DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Context

CMSC 124 Lab 4

Prepared by: Rene Andre Bedonia Jocsing

Published: October 22, 2026 **Deadline:** December 09, 2026

Your interpreter evaluates expressions, which makes it an unusually expensive calculator. Everything it computes vanishes the moment it's printed. This activity gives it a memory.

Two ideas do that. **Statements** let a program do things in sequence. **State** lets it remember things between them. Together they turn your language from a toy into something you could write a program in. This is the activity where most groups finally feel like they're building a language. It follows Chapter 8 of Nystrom's *Crafting Interpreters*, "Statements and State."

Your Language, Your Rules

The examples follow Lox, the language in the textbook. Your language can be imperative, functional, or a mixture. An **imperative** language changes stored values through commands. A **functional** language computes with expressions and functions, often keeping each binding unchanged. Both can meet this activity's requirements.

You need named bindings, lookup, nested scopes or binding forms, and observable program results. If your language supports mutation, implement assignment. If it keeps bindings immutable, demonstrate nested rebinding and shadowing without changing the outer value. A `let` expression can group computations and introduce bindings without a separate statement node.

Document whether lookup follows enclosing declarations (**static**, or lexical, scope), active callers (**dynamic** scope), or a stated combination. Blocks alone won't distinguish these rules. Lab 5's function calls will. Explain which binding wins when identifiers repeat.

Keep **scope**, the rule for visibility, separate from **lifetime**, the interval during which storage exists. **Binding time** is when a connection to a type, storage cell, or value is established. You can choose static storage, which lasts for the program, stack-dynamic storage, which lasts for an activation, or heap-dynamic storage. Explicit heap-dynamic storage uses language operations to allocate and release it. Implicit heap-dynamic storage binds storage through operations such as assignment and reclaims it according to the language's rules. Implement the categories your design uses. You don't need all four.

Describe the storage your interpreted language promises. A host-language map allocated on the heap doesn't, by itself, make every guest binding heap-dynamic. State when each binding becomes visible, when its storage begins and ends, and whether its value or type can change.

The Lox grammar and environment sketches below are worked examples. Adapt them to your documented rules. The run contract, earlier flags, harness annotations, and CI requirements still apply. Every alternative needs committed tests with predicted results. A paragraph defending a design doesn't replace a passing harness test.

Background

Expressions Against Statements

Everything you've implemented so far has been an expression: a piece of code that evaluates to a value. `2 + 3`, `"hello" + " world"`, and `!(x > 5)` are all expressions. Evaluate one and you get a value back.

A **statement** produces an effect. Statements are the verbs of a language. They make things happen. In most C-family languages, and probably yours, you turn an expression into a statement by putting a semicolon after it. `2 + 3;` is an expression statement. It evaluates the expression, throws the result away, and moves on. Useless by itself, but once expressions can have side effects, which they can as soon as assignment exists, expression statements are how most code does its work.

The Lox route adds print statements, variable declarations, and blocks this month. The functional route adds output or program results, immutable bindings, and nested binding expressions. Control flow and functions arrive in Lab 5.

Print Statements

Start with the simplest one. It's the one that makes everything else observable:

```
print "Hello, world!";
print 2 + 3;
print x + y;
```

When the interpreter meets a print statement, it evaluates the expression and displays the value. Your grammar grows a rule:

```
statement → exprStmt | printStmt ;
printStmt → "print" expression ";" ;
exprStmt  → expression ";" ;
```

Building `print` into the grammar as a statement is a slightly odd choice. Nystrom is candid that he does it because Lox has no functions yet and he needs to see output. You need observable output this month too. A print statement, a built-in output operation, or a documented program-result convention can provide it. In Lab 5, once your language has functions, you can decide whether to keep the statement or demote printing to a native function.

Variables and Environments

Variables are how a program remembers. They bind names to values that outlive the statement that produced them:

```
var x = 10;
var y = 20;
print x + y;
```

The data structure holding those bindings is an **environment**, sometimes called a context. The Lox version has two fields:

```
Environment {
    values      // a map from names to values
    enclosing   // the environment this one sits inside, or nothing
}
```

The map holds the local bindings. In this Lox sketch, `enclosing` links to the surrounding scope. Another design can use another representation.

One field buys you so much because of what it makes possible. Take this program, paused at the moment `print x` runs:

```
var x = "outer";
var y = 1;
{
    var x = "inner";
    print x;
}
```

At that instant, two environments exist, chained:

```
 inner block            global
+-------------+        +-------------+
| x = "inner" | -----> | x = "outer" |
|             | encl.  | y = 1       |
+-------------+        +-------------+
                        enclosing: none
```

Both environments have an `x`. That's fine. It's the point. Asking the inner one for `x` finds `"inner"` and stops, never consulting the outer. Asking the inner one for `y` doesn't find it locally, so the lookup follows `enclosing` outward and finds `1` in the global environment. Asking for `z` walks the same path, runs out of environments, and reports an undefined variable.

So a name that exists in several environments resolves to the innermost one holding it. A name that exists in none is an error. Both behaviors fall out of one chain walk. This sketch implements Lox's block lookup. Function lookup needs the scope decision you'll exercise in Lab 5.

Declaration creates a binding:

```
var identifier;
var identifier = initializer;
```

```
declaration → varDecl | statement ;
varDecl     → "var" IDENTIFIER ( "=" expression )? ";" ;
```

In Lox, executing one means evaluating the initializer if there's one, substituting your language's nil if there isn't, and defining the name in the current environment. Whether redeclaring an existing name in the same scope is legal is your call. Lox allows it at global scope, which is convenient in a REPL and confusing in a file. Decide, document, and test whichever you choose.

Access reads a binding, and assignment writes one:

```
identifier = expression;
```

```
expression → assignment ;
assignment → IDENTIFIER "=" assignment | equality ;
```

In Lox, assignment is an expression. That's why `var result = (x = 15);` works and why the rule recurses on the right. It evaluates the right side, finds the variable, errors if it doesn't exist, updates it, and produces the assigned value.

The Lox route depends on the distinction between introducing a binding and updating one.

In Lox, declaring and assigning search in opposite ways. Declaring with `var` always creates a binding right here, in the current environment, without looking anywhere else. Assigning with = never creates anything. It searches outward along the enclosing chain for a binding that already exists, and fails if it can't find one.

That asymmetry is what makes both shadowing and mutation work at once:

```
var x = "outer";
{
    var x = "inner";     // DECLARE: makes a new x here, outer x untouched
    x = "changed";       // ASSIGN: finds the inner x, changes it
}
```

```
var x = "outer";
{
    x = "changed";       // ASSIGN: no local x, walks out, changes the outer one
}
print x;                 // changed
```

Same two lines of source, opposite effects on the outer binding, and the only difference is the keyword. Blur the two in your implementation, by letting assignment fall back to creating a binding when it can't find one, and the second example makes a *new* local `x`, the outer one keeps saying `"outer"`, and every misspelled variable name in your language becomes a fresh variable. That's a bug when your specification says assignment only updates existing bindings. If assignment intentionally creates bindings, document where it creates them and test that rule. An immutable language instead tests rebinding and rejects mutation where its syntax permits an attempt.

Blocks and Scope

In Lox, a block groups statements and introduces a **scope**, a region of the program where a set of names is visible:

```
var x = "outer";
{
    var x = "inner";
    print x;   // inner
}
print x;       // outer
```

```
statement → exprStmt | printStmt | block ;
block     → "{" declaration* "}" ;
```

Entering a block creates a new environment whose enclosing reference points at the current one. Leaving it restores the previous environment. Whether the storage can be reclaimed depends on the lifetime and capture rules. The inner `x` doesn't overwrite the outer `x`. It **shadows** it. The outer one is still there when the block ends.

Lookup walks the chain. Ask an environment for a name, and if it doesn't have it, it asks the environment that encloses it, and so on until either someone has it or you run out of environments and report an

undefined variable. The links in this example follow source nesting, which gives lexical scope. Walking a chain is a technique. The relation represented by its links determines the scope rule. Lab 5 compares lexical links with active-caller links.

Script Mode

In the Lox route, running a file executes statements in order, and only print statements produce output. An expression-based language can instead evaluate top-level forms and display results according to its documented file convention. That's a change from Lab 3, where evaluating a file printed every expression's value, and it's the reason your `--eval` flag exists as a separate entry point.

The REPL keeps the friendlier behavior. An expression typed at the prompt prints its value, because a REPL where `2 + 3` prints nothing is a bad REPL. Most implementations handle this by checking, in the REPL only, whether what was parsed is a bare expression statement and printing its value if so.

Learning Objectives

By the end of this activity, you should be able to:

- Explain how your language groups computations and produces observable results
- Extend the parser to accept programs with named bindings and nested scopes or binding forms
- Implement binding creation and lookup, plus assignment when mutation is supported
- Explain shadowing and your chosen lookup rule using a concrete trace
- Distinguish visibility, storage lifetime, and binding time
- Distinguish a static error from a runtime error for name resolution, and choose exit codes accordingly
- Recover at statement or expression-form boundaries after a syntax error
- Explain file and REPL behavior while sharing the evaluator

Task

Extend your interpreter to execute programs.

The tested path is now plain `./run <path-to-source-file>`, with no flag. It parses and evaluates a program according to your grammar, including its declarations, statements, or expression forms. The demonstrated path is the REPL, which accepts your program forms and expressions. Your `--tokenize`, `--parse`, and `--eval` flags all stay, and `tests/lab1` through `tests/lab3` have to keep passing.

Some housekeeping comes with that. `tests/lab0/hello.<ext>` was allowed to be nonsense back when `./run` ignored its argument. Now that plain `./run` interprets, that file has to be a program in your language that prints what `hello.expected` says. Rewrite it using your output operation or program-result convention, with the expected output. It's a satisfying moment. The first thing you ever committed to the repository now runs through your own interpreter.

Required Features

Grammar Extension

Add the forms your language uses for output, binding creation, lookup, and nested scope. Add assignment if mutation is supported. An immutable language can use `let` expressions and expression bodies. Update the Grammar section of your `README.md`. Explain where bindings and other forms are legal.

Statement Parser

Extend your parser to produce the statement or expression nodes your design needs. It must parse programs, nest scopes or binding forms to arbitrary depth, and report clear errors for malformed forms and missing delimiters.

Synchronization means discarding tokens after an error until a new statement or expression form can plausibly begin. Report the recovered syntax errors, then exit 65 without executing anything. Use your grammar's boundaries, such as terminators, closing delimiters, or binding keywords.

If assignment exists, check that its target is assignable. The Lox route parses the left expression before seeing = and then validates the target. If bindings are immutable, reject attempted mutation where your syntax can express it. Document whether that rejection happens before or during execution.

Environment

Implement binding creation and lookup according to your scope rule. Add an update operation if bindings can change. Define, get, and assign are the Lox sketch's operations, but your implementation can use other names and structures.

Document where new bindings appear, how lookup selects one, and what rebinding or assignment does. Test the difference between introducing an inner binding and changing an existing one. An immutable language must preserve the outer binding during inner rebinding.

Statement Evaluator

Evaluate your program forms and produce output according to the file convention. Lox discards expression-statement values, prints explicitly, and executes declarations and blocks. A functional language can evaluate `let` bodies and expose their results through output or the program-result convention.

Restore the active evaluation context after a nested form ends, including when evaluation raises an error. This keeps the REPL usable. Restoring visibility doesn't require destroying storage that your lifetime rules retain. Test both facts separately.

Undefined Variables

In the Lox route, reading or assigning a name that was never declared is a runtime error: message with a line number on stderr, exit 70 in file mode, prompt returns in the REPL.

This is a decision. A language could catch undefined names before execution starts and exit 65. Many do, since the information is available statically. Lox chooses runtime, partly because the check is simpler and partly because it wants recursive and mutually recursive functions to work in Lab 5 without a separate resolution pass. Chapter 11 of the textbook, "Resolving and Binding," is where Nystrom revisits this. You aren't required to read it. If you decide to catch undefined names statically instead, say so in your `README.md`, use exit 65, and be ready to explain how you'll keep mutual recursion working next month.

Script Execution and the REPL

`./run <file>` executes a file. `./run` with no arguments starts the REPL, which accepts binding forms and supported statements, keeps state across lines, and prints the value of a bare expression. A runtime error in the REPL prints and returns the prompt. The session's variables survive.

The Run Contract

Plain `./run <file>` finally means what Lab 0 said it meant:

```
./run           tests/lab4/scope.mylang          # execute a program
./run --tokenize tests/lab1/keywords.mylang      # Lab 1
```

```
./run --parse    tests/lab2/precedence.mylang     # Lab 2
./run --eval     tests/lab3/arithmetic.mylang     # Lab 3
```

Exit 0 when the program ran, 65 when you rejected it before running anything, 70 when it started and hit a runtime error. Diagnostics on stderr, program output on stdout, as always.

Writing Tests

Inline mode again, and this time with no flag at all, since plain execution is the thing under test. `tests/lab4/manifest.json`:

```
{
  "ext": ".mylang",
  "mode": "inline",
  "comment_prefix": "//"
}
```

The output and error annotations stay. Use `expect nothing` when a file intentionally produces no output. `expect:` checks stdout line by line in order, `expect runtime error:` checks a substring of stderr and exit 70, and `expect error:` checks a substring of stderr and exit 65.

Statements make inline tests better than they were in Lab 3, because now a single file can tell a story. `tests/lab4/shadowing.mylang`:

```
var x = "outer";
{
    var x = "inner";
    print x;          // expect: inner
}
print x;              // expect: outer
```

These lines check two properties of Lox block scope. The inner declaration didn't destroy the outer binding. The outer binding came back when the block ended. Compare that with the two-file sidecar version and you can see why the textbook works this way.

An immutable language can prove the same separation with a binding expression. This illustrative syntax uses `let ... in ...` and a built-in output operation. Translate it to your grammar. `tests/lab4/immutable-shadowing.mylang`:

```
let x = "outer" in {
    print(let x = "inner" in x);  // expect: inner
    print(x);                     // expect: outer
}
```

The inner binding changes which `x` the inner body sees. It leaves the outer value intact. No assignment is needed. Use your manifest's comment syntax for the annotations. The harness still checks the two stdout lines in order through plain `./run`.

On the mutable Lox route, nesting and assignment through a scope chain deserve their own file, `tests/lab4/nested.mylang`:

```
var a = 1;
{
    var b = 2;
    {
        a = a + b;    // assignment reaches two scopes up
        print a;      // expect: 3
    }
    print b;          // expect: 2
}
print a;              // expect: 3
```

That test would pass even if your `assign` wrongly created a new local binding, right up until the last line. Which is why the last line is there.

Errors, one per file:

```
print a;              // expect runtime error: Undefined variable 'a'.
```

```
var x = 1
print x;              // expect error: Expect ';' after variable declaration.
```

Annotate the message text only, never a `[line N]` prefix, since the stderr check is a substring match and the prefix is the part most likely to change.

Run every folder locally before pushing:

```
curl -sSL https://raw.githubusercontent.com/WhiteLicorice/cmsc-124-harness/v1.1/run_tests.py
  ↪  -o run_tests.py
./build.sh
python run_tests.py tests/lab0
python run_tests.py tests/lab1
python run_tests.py tests/lab2
python run_tests.py tests/lab3
python run_tests.py tests/lab4
```

Use `python3` on Linux and macOS, and Git Bash on Windows. Fifteen seconds per file still applies. This is the first activity where an accidental infinite loop is plausible. A parser that fails to consume a token during synchronization will happily spin until the harness kills it.

Continuous Integration

```
      - run: python3 run_tests.py tests/lab0
      - run: python3 run_tests.py tests/lab1
      - run: python3 run_tests.py tests/lab2
      - run: python3 run_tests.py tests/lab3
      - run: python3 run_tests.py tests/lab4
```

Before you book a defense, all five folders must be green. The `tests/lab0` line is the one to watch this month, since it's the folder whose meaning just changed underneath you.

Implementation Notes

The Environment Chain

The Lox environment holds a map and an enclosing reference, which is null or nil for the global one. Here is its binding implementation:

```
function define(name, value):
    values[name] = value              // always local, never walks

function get(name):
    if values has name : return values[name]
    if enclosing exists : return enclosing.get(name)     // walk outward
    throw RuntimeError(name, "Undefined variable '" + name + "'.")

function assign(name, value):
    if values has name:
        values[name] = value
        return
    if enclosing exists:
        enclosing.assign(name, value)                    // walk outward
        return
    throw RuntimeError(name, "Undefined variable '" + name + "'.")
```

Look at what's present and what's absent. `get` and `assign` each recurse outward and each throw when they run out of environments. `define` does neither. It has no `enclosing` branch at all, and adding one would violate this explicit-declaration design. Check those branches against the scope and mutation rules you documented.

Note also that `define` overwrites. That's what makes redeclaration in the same scope legal, so if your design forbids it, this is the one line where you check first and complain.

Watch the ownership question if your host language cares about it. In Rust and C++, the natural implementation is a reference-counted pointer with interior mutability, if your design retains bindings for escaping closures in Lab 5. Reference counting also needs a plan for cycles. Choose ownership that matches your lifetime rules.

Executing a Block

In the Lox sketch, save the current environment, install a new one enclosing it, execute the statements, and restore the saved one no matter how you leave:

```
function executeBlock(statements, blockEnvironment):
    previous = this.environment
    try:
        this.environment = blockEnvironment      // install
        for statement in statements:
            execute(statement)
    finally:
        this.environment = previous              // restore, always
```

The `finally` isn't decoration. Without it, any runtime error inside the block, a division by zero or an undefined variable, propagates upward while leaving `this.environment` pointing at a scope that has conceptually ended. In file mode you might not notice, since the process is about to exit anyway. In the REPL you absolutely will. The session keeps running. Every subsequent line evaluates inside a dead block scope, and variables you declare afterward vanish into it. Use whatever your host gives you, `finally`, `defer`, a scope guard, or `Drop`, but use something.

Uninitialized Variables

If your grammar permits `var x;`, it has to mean something. Nil is the usual answer and it's the simplest. The alternative is a distinct "declared but not initialized" state that errors on read, which catches bugs at the cost of more machinery. Either is fine. Either goes in your `README.md`.

Two Modes, One Interpreter

The interpreter instance should be shared across a REPL session so state persists. File mode creates one, runs the program, and exits with the right code. REPL mode creates one and feeds it lines. The REPL prints bare-expression values and continues after errors. File output follows your documented program convention. If you find yourself with two evaluators, one of them will drift and it'll be the untested one.

Common Pitfalls

- Assignment creating a binding when the specification requires an existing one
- Inner rebinding changing the outer value in an immutable design
- Lookup following a different relation from the documented scope rule
- Treating scope exit as storage destruction when capture or static storage keeps it alive
- Failing to restore the evaluation context after an error
- Accepting invalid binding or assignment targets
- Synchronization that consumes no tokens, giving a 15-second timeout
- Missing delimiters reported with messages unrelated to the malformed form
- Leaving `tests/lab0/hello` as nonsense after plain `./run` begins interpreting
- Using exit 65 for a documented runtime error, or 70 for a documented static error

---

Testing Strategy

Your `tests/lab4/` folder must cover binding creation and lookup, nested scopes or binding forms at least two deep, shadowing or the documented repeated-identifier rule, and observable results. Test which outer bindings remain accessible after an inner form ends.

If mutation exists, test assignment and its effect on the selected binding. Include assignment inside an expression if the grammar permits it. If bindings are immutable, test nested rebinding and preservation of the outer value. Test declarations without initializers only if they exist.

Then test errors, one per file: an unresolved name, malformed binding syntax, an unclosed grouping or scope, and any illegal redeclaration, assignment, or immutable update your grammar can express. Use the error annotation matching your chosen static or runtime phase.

For a form that intentionally produces no output, use `expect nothing`. If successful file evaluation always prints a result, document that rule and test its output instead. Never submit an annotation-free file as proof of silence.

Trace each binding test by hand first. Draw the lookup path, predict the output, write the annotations, and run `python run_tests.py tests/lab4`. A drawing supports the committed test. It doesn't replace it.

Deliverables

Your group's GitHub repository, presented during appointments, with a clean incremental commit history showing who wrote what, must contain:

1. Program-form nodes, the binding implementation, and the evaluation logic, documented where the code doesn't explain itself
2. A working scanner, parser, and expression evaluator, with all three earlier flags intact
3. `build.sh` and `run` at the root, executable, where plain `./run <file>` executes a program and `./run` starts a REPL
4. `tests/lab4/` with `manifest.json` and your annotated test files, plus an updated `tests/lab0/hello.<ext>` that's a program in your language
5. `.github/workflows/test.yml` running all five lab folders, green on the commit you defend
6. A `README.md` with the extended grammar, plus the specification template's paradigm, mutability, scope, binding-time, storage-lifetime, output, and undefined-name decisions
7. At least one example program under an `examples/` directory that uses variables and scope to do something you'd want to do

Then, each member submits, individually after the laboratory defense through email: a short `reflection.txt` covering what broke, what you fixed, and what you learned, plus a short `peer.txt` with your honest assessment of how your groupmates, including yourself, worked during the activity. Adhere to the following subject line: `[CMSC 124 Lab] Lab 4: LastName, Initials`, for example: `[CMSC 124 Lab] Lab 4: Sanchez, SM`. Include a link to your group's GitHub repository in the email. If even one member of a group fails to submit their individual deliverables, no final grade for the activity may be released for all members of the group.

Academic Honesty

Using large language models to generate wholesale vibe-coded submissions is cheating, subject to failure in the course and harsh disciplinary action. Bindings are where generated code goes wrong in ways the author can't explain. Expect to trace lookup, rebinding, and storage lifetime through your own code.

Important Dates

Progress reports and the laboratory defense may be booked only during the dates and hours defined in the syllabus. Book ahead through the booking page on the course site.

Activity	Monday	Tuesday
Week 1 Progress Report	Oct 26	Oct 27
Week 2 Progress Report	Nov 2	Nov 3
Week 3 Progress Report	Nov 9	Nov 10
Week 4 Laboratory Defense	**Nov 16**	**Nov 17**

Appointments need my verification before they count. If a date falls on a holiday or a suspension of classes, the syllabus allows a recorded progress report or defense for that seven-day period.

For progress reports, Week 1 wants observable program results from a file and the updated grammar. Week 2 wants binding creation and lookup, plus assignment or immutable rebinding, with harness tests alongside the code. Week 3 wants nested scopes or binding forms, shadowing or the documented alternative, synchronized error recovery, green CI across all five folders, and your documented decisions.

This is the last activity with a comfortable margin. Lab 5 is the largest of the five. Its defense window is the last one before the end of classes, and finishing this one on schedule is what buys you room next month.

Grading Rubric

Criteria	Excellent (90-100%)	Good (75-89%)	Fair (60-74%)	Poor (0-59%)
Implementation Correctness (25%)	Every required capability works under the documented language design, including edge cases; the run contract is honored exactly, with program output on stdout, diagnostics on stderr, and the right exit code for each class of failure.	Core features correct with minor gaps in fringe cases; streams and exit codes right.	Core features work, but several requirements are missing or wrong; stream separation or exit codes are inconsistent.	Fails the activity's core requirement, ignores the run contract, or crashes with host-language errors on malformed input.
Testing and CI (20%)	Committed tests cover every required feature and its failure cases; expectations were reasoned out; every lab folder is green in CI on the defended commit.	Good coverage of the main features with at least one failure case; CI green.	Only happy paths tested, or an earlier lab folder left failing.	Almost no tests, no manifest, or no working workflow.
Code Engineering (10%)	Clean separation between pipeline stages; shared logic written once; constructs idiomatic for the host language; comments explain why.	Mostly clean structure with minor duplication or naming problems.	Monolithic functions; repeated logic; one stage leaking into another.	Unreadable, or the code contradicts the documented language design.
Collaboration (20%)	Professional Git usage: atomic, semantic commits from every member spread across the accomplishment period; tests and features arrive together; evidence of review or pair programming.	Consistent use of version control; adequate commit messages; evidence of a structured team workflow.	Inconsistent Git usage; large code dumps instead of incremental progress; vague commit messages.	Minimal use of version control; repository lacks history, or history was rewritten or crammed.
Technical Defense (25%)	Every member traces their own code over fresh input, handles what-if questions, and defends the language design decisions behind the implementation.	Clear explanation of the implementation; both members participate meaningfully; logic delivery is sound.	Unclear explanations; uneven participation; struggles with code-tracing questions.	Unprepared; cannot explain the pipeline or defend design decisions.